

Chapter 1

Introduction to real-time systems

1.1	Definition of a real-time system	1.4	Development cycle for real-time systems
1.2	Examples of real-time systems	1.5	Languages for programming real-time systems
1.3	Characteristics of real-time systems		Summary
			Further reading
			Exercises

As computers become smaller, faster, more reliable and cheaper, so their range of application widens. Built initially as equation solvers, their influence has extended into all walks of life, from washing machines to air traffic control. One of the fastest expanding areas of computer exploitation is that involving applications whose prime function is *not* that of information processing, but which nevertheless require information processing in order to carry out their prime function. A microprocessor-controlled washing machine is a good example of such a system. Here, the prime function is to wash clothes; however, depending on the type of clothes to be washed, different ‘wash programs’ must be executed. These types of computer applications are generically called **real-time** or **embedded**. It has been estimated that 99% of the worldwide production of microprocessors is used in embedded systems. These embedded systems place particular requirements on the computer languages needed to program them, as they have different characteristics from the more traditional information processing systems.

This book is concerned with embedded computer systems and their programming languages. It studies the particular characteristics of these systems and discusses how modern real-time programming languages and operating systems have evolved. In order to give the chapters continuity, three programming languages are considered in detail: Ada, Java and C. These languages have been chosen because they are actually used for software production. As C is a sequential language, it is used in conjunction with the POSIX family of operating system interfaces (in particular, the real-time extensions). To emphasize this, it will be referred to a *C/Real-Time POSIX*. As Java was not originally intended to be used for real-time systems development it must be augmented with the facilities of the Real-Time Specification for Java (RTSJ). To emphasize

this, it will be referred to as *Real-Time Java*. Ada was designed for systems development; however, some of its libraries (for example, the `Real_Time` package) are optional. Here we assume these libraries are supported.

1.1 Definition of a real-time system

Before proceeding further, it is worth trying to define the phrase ‘real-time system’ more precisely. There are many interpretations of the exact nature of a real-time system; however, they all have in common the notion of response time – the time taken for the system to generate output from some associated input. The *Oxford Dictionary of Computing* gives the following definition of a real-time system:

Any system in which the time at which output is produced is significant. This is usually because the input corresponds to some movement in the physical world, and the output has to relate to that same movement. The lag from input time to output time must be sufficiently small for acceptable timeliness.

Here, the word ‘timeliness’ is taken in the context of the total system. For example, in a missile guidance system, output is required within a few milliseconds, whereas in a computer-controlled car assembly line, the response may be required only within a second. To illustrate the various ways in which ‘real-time’ systems are defined, two further definitions will be given. Young (1982) defines a real-time system to be:

any information processing activity or system which has to respond to externally generated input stimuli within a finite and specified period.

Another definition is (Randell et al., 1995):

A real-time system is a system that is required to react to stimuli from the environment (including the passage of physical time) within time intervals dictated by the environment.

In their most general sense, all these definitions cover a very wide range of computer activities. For example, an operating system like Windows may be considered real-time in that when a user enters a command he or she will expect a response within a few seconds. Fortunately, it is usually not a disaster if the response is not forthcoming. These types of system can be distinguished from those where *failure* to respond can be considered just as bad as a wrong response. Indeed, for some, it is this aspect that distinguishes a real-time system from others where response time is important but not crucial. Consequently, *the correctness of a real-time system depends not only on the logical result of the computation, but also on the time at which the results are produced*. Practitioners in the field of real-time computer system design often distinguish between **hard** and **soft** real-time systems. Hard real-time systems are those where it is absolutely imperative that responses occur within the specified deadline. Soft real-time systems are those where response times are important but the system will still function correctly if deadlines are occasionally missed. Soft systems can themselves be distinguished from

interactive ones in which there are no explicit deadlines. For example, the flight control system of a combat aircraft is a hard real-time system because a missed deadline could lead to a catastrophe, whereas a data acquisition system for a process control application is soft, as it may be defined to sample an input sensor at regular intervals but to tolerate intermittent delays. Of course, many systems will have both hard and soft real-time sub-systems. Indeed, some services may have both a soft and a hard deadline. For example, a response to some warning event may have a soft deadline of 50 ms (for an optimally efficient reaction) and a hard deadline of 200 ms (to guarantee that no damage to equipment or personnel takes place). Between 50 ms and 200 ms, the 'value' (or utility) of the output decreases.

As these definitions and examples illustrate, the use of the term 'soft' does not imply a single type of requirement, but incorporates a number of different properties. For example:

- the deadline can be missed occasionally (typically with an upper limit of misses within a defined interval);
- the service can occasionally be delivered late (again, with an upper limit on tardiness).

A deadline that can be missed occasionally, but in which there is no benefit from late delivery, is called **firm**. In some real-time systems, optional firm components may be given probabilistic requirements (for example, a hard service must produce an output every 300 ms; at least 80% of the time this output will be produced by a firm component, X; on other occasions, a hard, but functionally much simpler component, Y, will be used).

In this book, the term 'real-time system' is used to mean both soft and hard real-time. Where discussion is concerned specifically with hard real-time systems, the term 'hard real-time' will be used explicitly.

In a hard or soft real-time system, the computer is usually interfaced directly to some physical equipment and is dedicated to monitoring or controlling the operation of that equipment. A key feature of all these applications is the role of the computer as an information processing component within a larger engineering system. It is for this reason that such applications have become known as **embedded computer systems**.

Another means of classifying the role that time has in real-time systems is to distinguish between **reactive systems** and **time-aware systems**. Time-aware systems make explicit references to the time frame of the enclosing environment. For example, if a bank safe's doors are to be locked from midnight to nine o'clock in the morning then these absolute time values must be available to the system. By comparison, a reactive system is typically concerned with relative times: an output has to be produced within 50 ms of an associated input. The key requirement of a reactive system is that it 'keeps up with the environment'. Often reactive systems are also control systems and hence they need to be synchronized with their environment. In particular, input sampling and output signalling must be done very regularly with controlled variability – there is a need to bound what is called **input jitter** and **output jitter**.

In order for a reactive systems to 'keep up with its environment' they are often structured to be **time-triggered**. All computation activities are **periodic** in that they have a defined cycle time, for example 50 ms, and are released for execution by an internal clock. The alternative to time-triggered is **event-triggered** in which the environment

explicitly controls (perhaps via an interrupt) the release for execution of some software activity. These activities are termed **aperiodic** or, if there is a bound on how often the releasing event can occur in any time interval, **sporadic**. Many systems will contain periodic and sporadic activities. However, some design approaches restrict the software architecture so that there are only time-triggered activities; ‘events’ must be polled for (that is, examined via a periodic activity).

Again, in this book a broad definition of ‘real-time system’ is assumed. The term is taken to include reactive and time-aware systems that may have both time-triggered and event-triggered invocations of work. Periodic, aperiodic and sporadic activities are all likely to be present in the same system.

1.2 Examples of real-time systems

Having defined what is meant by a real-time systems, some examples of their use are now given.

1.2.1 Process control

The first use of a computer as a component in a larger engineering system occurred in the process control industry in the early 1960s. Nowadays, the use of microprocessors is the norm. Consider the simple example shown in Figure 1.1, where the computer performs a single activity: that of ensuring an even flow of liquid in a pipe by controlling a valve. On detecting an increase in flow, the computer must respond by altering the valve angle; this response must occur within a finite period if the equipment at the receiving end of

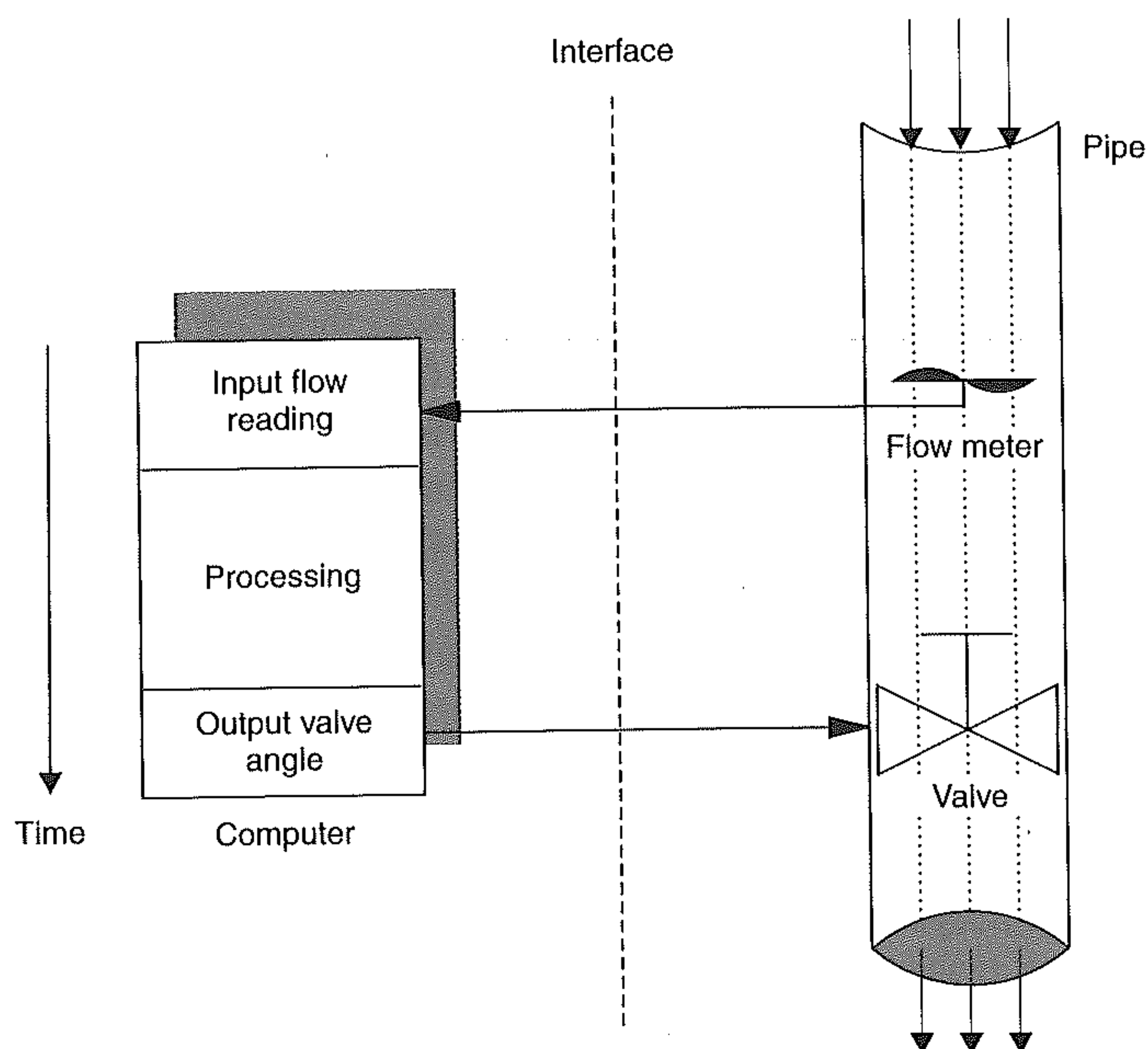


Figure 1.1 A fluid control system.

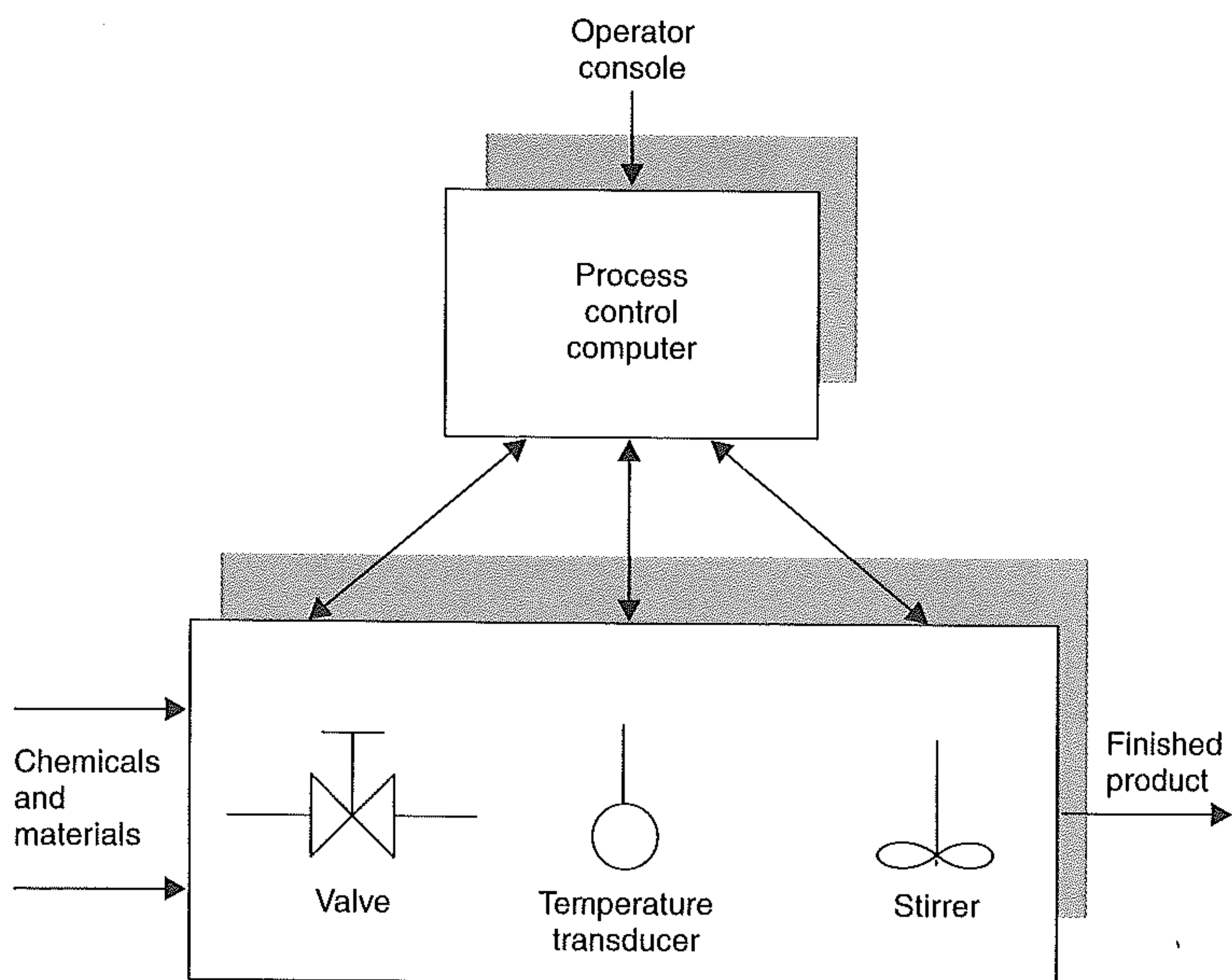


Figure 1.2 A process control system.

the pipe is not to become overloaded. Note that the actual response may involve quite a complex computation in order to calculate the new valve angle.

This example shows just one component of a larger control system. Figure 1.2 illustrates the role of a real-time computer embedded in a complete process control environment. The computer interacts with the equipment using sensors and actuators. A valve is an example of an actuator, and a temperature or pressure transducer is an example of a sensor. (A transducer is a device that generates an electrical signal that is proportional to the physical quantity being measured.) The computer controls the operation of the sensors and actuators to ensure that the correct plant operations are performed at the appropriate times. Where necessary, analog-to-digital (and digital-to-analog) converters must be inserted between the controlled process and the computer.

1.2.2 Manufacturing

The use of computers in manufacturing has become essential in order that production costs can be kept low and productivity increased. Computers have enabled the integration of the entire manufacturing process from product design to fabrication. It is in the area of production control that embedded systems are best illustrated. Figure 1.3 diagrammatically represents the role of the production control computer in the manufacturing process. The physical system consists of a variety of mechanical devices – such as machine tools, manipulators and conveyor belts – all of which need to be controlled and coordinated by the computer.

A modern manufacturing control system will employ a wide range of robots. These will again need to be controlled and coordinated, but they are also autonomous real-time

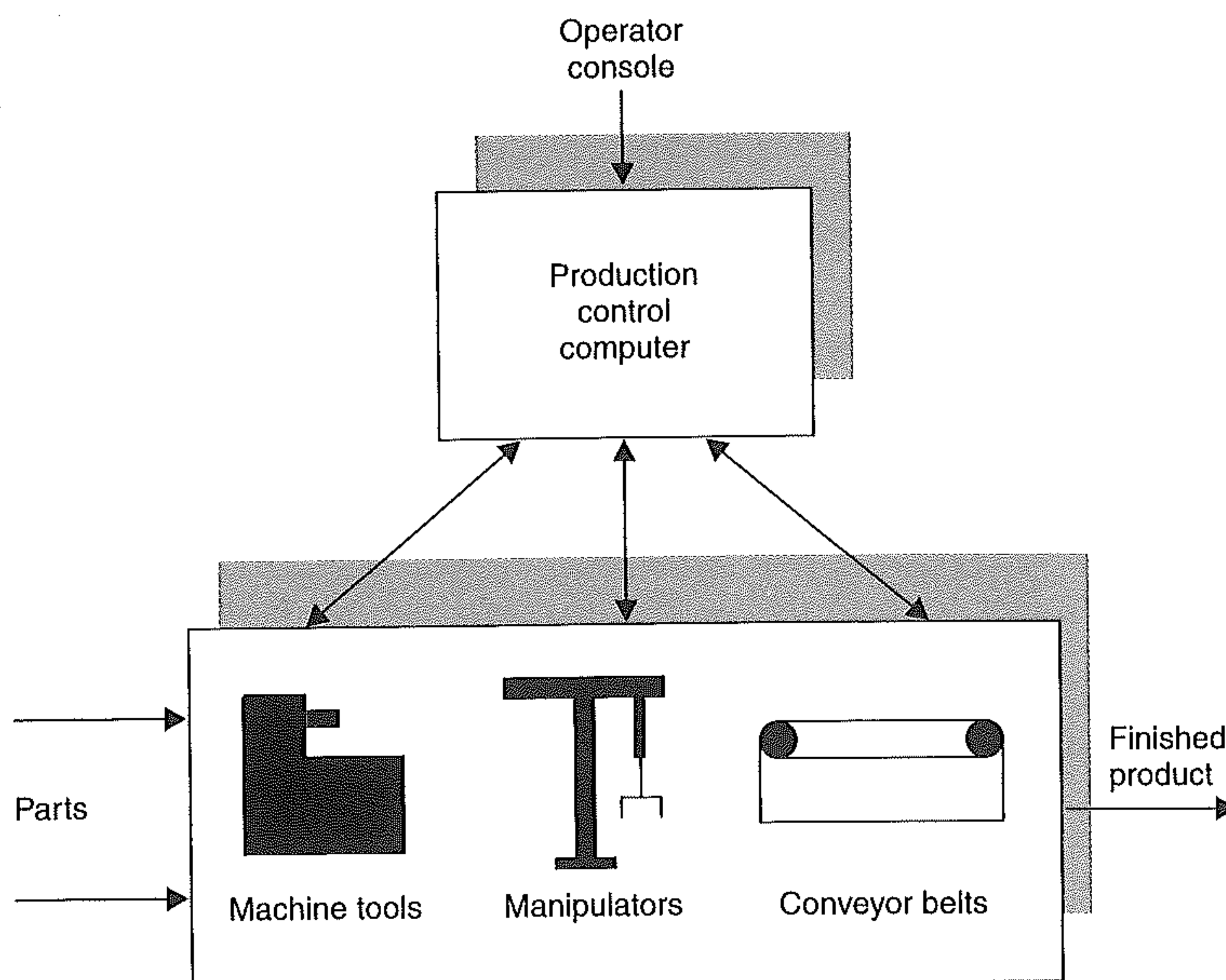


Figure 1.3 A production control system.

systems in their own right. They have large numbers of sensors (for example proximity indicators), many moving parts that need controlling and often vision subsystems that require considerable computational power. When mobile robots and humans operate in the same physical space then there are considerable safety issues that dictate that at least part of the robots' functions are hard real-time.

1.2.3 Communication, command and control

Although **communication, command and control** is a military term, there is a wide range of disparate applications which exhibit similar characteristics; for example, airline seat reservation, medical facilities for automatic patient care, air traffic control, remote bank accounting and large-scale manufacturing plants. Each of these time-aware systems consists of a complex set of policies, information gathering devices and administrative procedures which enable decisions to be supported, and provide the means by which they can be implemented. Often, the information gathering devices and the instruments required for implementing decisions are distributed over a wide geographical area. Figure 1.4 diagrammatically represents such a system.

1.2.4 A typical embedded real-time system

In each of the examples shown above, the computer is interfaced directly to physical equipment in the real world, and is reacting to changes in this environment. In order to control these real-world devices, the computer will need to sample the measurement devices at regular intervals (i.e. periodic activities); a real-time clock is therefore required.

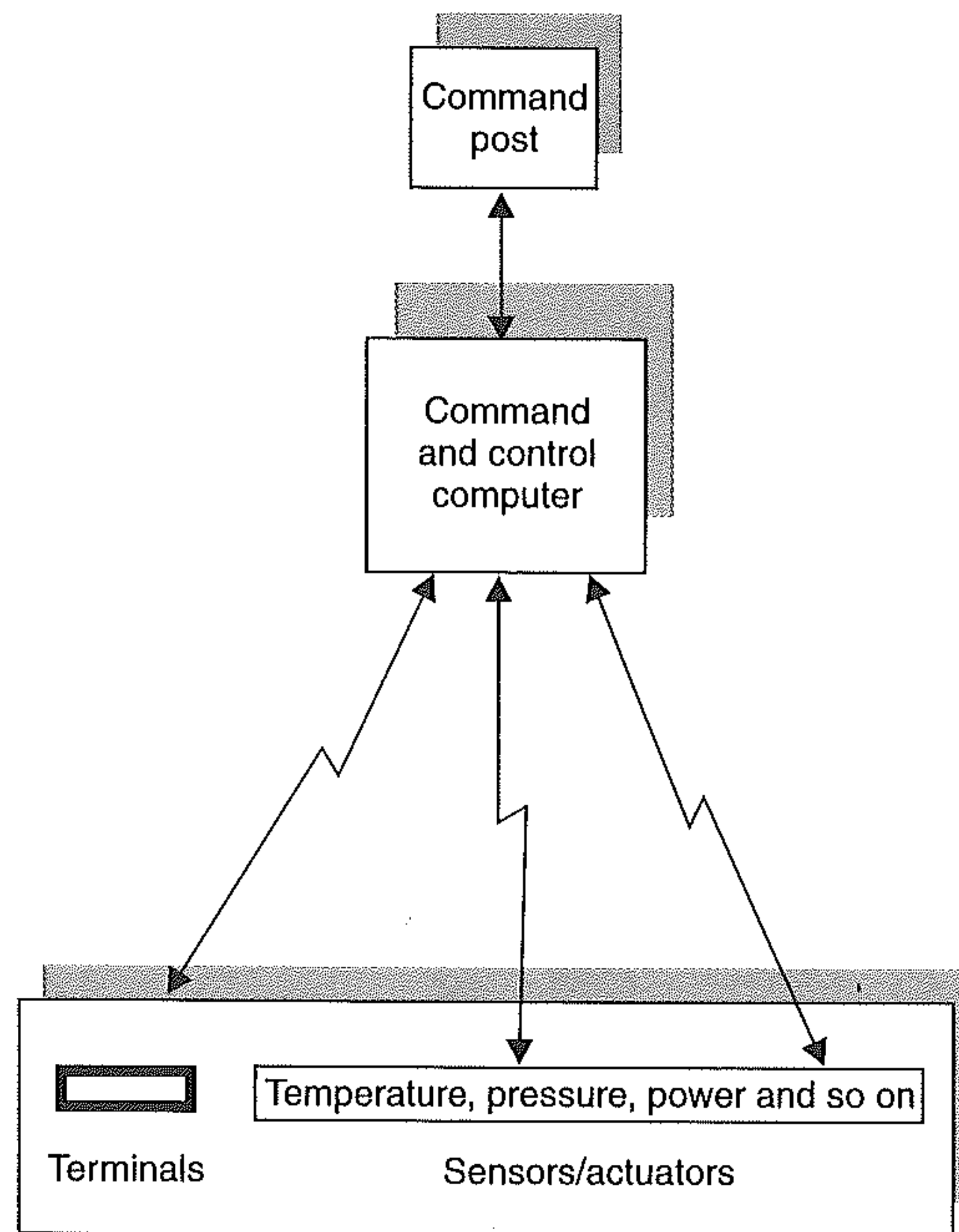


Figure 1.4 A command and control system.

Usually there is also an operator's console to allow for manual intervention. The human operator is kept constantly informed of the state of the system by displays of various types, including graphical ones.

Records of the system's state changes are also kept in a database which can be interrogated by the operators, either for a post mortem (in the case of a system crash), or to provide information for administrative purposes. Indeed, this information is increasingly being used to support decision making in the day-to-day running of systems. For example, in the chemical and process industries, plant monitoring is essential for maximizing economic advantages rather than simply maximizing production. Decisions concerning production at one plant may have serious repercussions for other plants at remote sites, particularly when the products of one process are being used as raw material for another.

A typical large embedded real-time computer system can, therefore, be represented by Figure 1.5. The software which controls the operations of the system can be written in modules which reflect the physical nature of the environment. Usually there will be a module which contains the algorithms necessary for physically controlling the devices; a module responsible for recording the system's state changes; a module to retrieve and display those changes; and a module to interact with the operator.

1.2.5 Multi-media systems

Entertainment systems such as radios, televisions, stereo systems, video systems and games of various kinds are all real-time systems in which the temporal requirements are

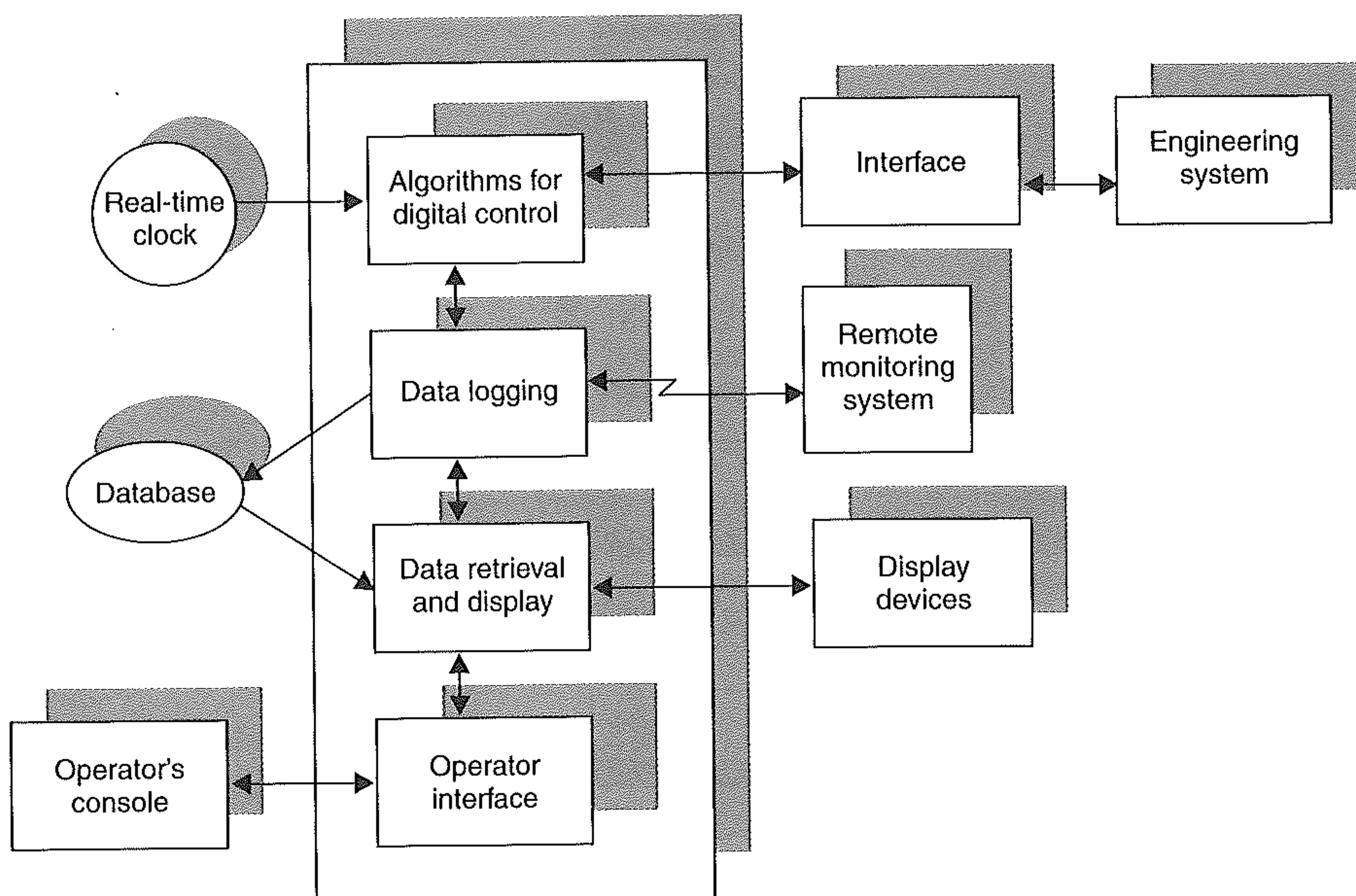


Figure 1.5 A typical embedded system.

determined by the perception and cognition of human users. There are clear requirements on visual frame rates, sound/picture synchronization and response times (to, for example, joystick movements) that must be satisfied if the quality of the human experience is not to be noticeably compromised. Multi-media systems in which many of these entertainment media are integrated with each other and with services such as phone communications, internet searches, email interactions, news updates, and the streaming of audio and video content from remote servers pose significant challenges for developers. There is an economic imperative to manufacture these systems at minimum cost, but the quality of the user experience must be maintained – and this means that the temporal requirements have to be central to the development process. From the design of special purpose on-chip units (for example, video decoders) to the choice of communication subsystems and the implementation of the software control components, the need to manipulate significant volumes of digital data within tight timing deadlines is the key engineering challenge.

The development of these home-based multi-media systems is one noticeable application area for real-time systems, but such fixed systems are not the only commodities under development. There is an increasing need to allow devices to be mobile whilst still linked to communication services. The mobile phone is evolving into a general-purpose essential accessory that supports most of the technologies of the fixed system together with new emerging services such as providing a means of paying for goods in shops, an aid to personal security and/or surveillance and of course a camera and general-purpose recording device. The one single defining property of all mobile systems is that they are indeed *mobile* – they rely on batteries for power. And batteries have finite life which adds to the challenges of developing these complex real-time systems.

1.2.6 Cyber-physical systems

A relatively new application area for real-time systems comes from the linking of digital information systems, such as that furnished by the internet, and real-time data collection typified by networks of sensors. This coupling of what is usually called the cyber world with the physical world has naturally led to the term *cyber-physical systems*. An example of a cyber-physical service is an adaptive navigation aid. Here there is a link between a detailed digitalized (and static) road map and traffic flow information that is being sensed at key locations on the roads. This traffic information is clearly real-time and dynamic. The navigational aid can potentially use sophisticated traffic flow models to predict future problems and advise as to the least congested route. This advice will diminish in usefulness as the data on which it is based becomes stale. Hence the need to update the advice as new data becomes available.

Cyber-physical systems may operate over wide areas, perhaps even globally, involve many computational elements and communication services, and have access to enormous volumes of real-time data. In the traffic system, consider the quantity of data generated if all junctions on all roads in Europe (or the USA or China) are sensing traffic flow (in all directions) and feeding this data into the internet. The sensing component of the cyber-physical system may be quite simple, for example a temperature sensor, or be a significant entity in its own right, such as a camera permanently viewing some scene. The local sensory system must, in this case, decide whether raw data or a processed (simpler) form of the data is fed into the digital world.

A cyber-physical system is likely to have real-time requirements that range from the millisecond level for the sensing activity to minutes or even hours for the applications that are built upon the amalgamation of the dynamic and static data. It should be clear to all readers, who have presumably spent time ‘on the net’, that the current internet with its simple protocols and architecture cannot guarantee real-time behaviour. Future enhancements (or alternatives) to the internet are, however, likely to respond to the challenges of cyber-physical systems and provide QoS (Quality of Service) that does provide real-time guarantees – even if the guarantees are not absolute (hard) but have a high level of probability associated with them.

1.3 Characteristics of real-time systems

A real-time system possesses many special characteristics (either inherent or imposed) which are identified in the following sections. Clearly, not all real-time systems will exhibit all these characteristics; however, any general-purpose language (and operating system) which is to be used for the effective programming of real-time systems must have facilities which support these characteristics.

1.3.1 Real-time facilities

Response time is crucial in any embedded system. Unfortunately, it is very difficult to design and implement systems which will guarantee that the appropriate output will be generated at the appropriate times under all possible conditions. It is often impossible to do this, and make full use of all computing resources at all times. For this reason, real-time systems are usually constructed using processors with considerable spare capacity,

thereby ensuring that ‘worst-case behaviour’ does not produce any unwelcome delays during critical periods of the systems’ operations.

Given adequate processing power, language and run-time support are required to enable the programmer to:

- specify times at which actions are to be performed;
- specify times by which actions are to be completed;
- support repeating (periodic or aperiodic) work;
- control (i.e. bound) the jitter on input and output operations;
- respond to situations where not all of the timing requirements can be met;
- respond to situations where the timing requirements are changed dynamically.

These are called real-time control facilities. They enable the program to synchronize with time itself. For example, with direct digital control algorithms it is necessary to sample readings from sensors at certain periods of the day, for example, 2 p.m., 3 p.m. and so on, or at regular intervals, for instance, every 5 seconds (with control systems, sample rates can vary from a few hundred hertz to several hundred megahertz). As a result of these readings, other actions will need to be performed. In an electric power station, it is necessary at 6 p.m. on Monday to Friday each week to increase the supply of electricity to domestic consumers. This is in response to the peak in demand caused by families returning home from work, turning on lights, cooking dinner and so on. In recent years in the UK, the demand for domestic electricity reaches a peak immediately after high-profile sporting events, when millions of viewers leave their living rooms, turn on lights in the kitchen and switch on the kettle in order to make a cup of tea or coffee.

An example of a dynamic change to the timing requirements of a system can be found in an aircraft flight control system. If an aircraft has experienced depressurization, there is an immediate need for all computing resources to be given over to handling the emergency. More normally, the moves from taxiing to taking off to climbing and then to cruising all involve changes to the basic operation of the flight control system. These changes, which are known generally as **mode changes**, also have consequences for the temporal characteristics of the executing software.

In order to meet response times, it is necessary for a system’s behaviour to be predictable. Chapters 9–12 consider the facilities and techniques used to obtain predictable program behaviour.

1.3.2 Concurrent control of separate system components

An embedded system will tend to consist of computers and several coexisting external elements with which the computer programs must interact simultaneously. It is the very nature of these external real-world elements that they exist in parallel. In our typical embedded computer example, the program has to interact with an engineering system (which will consist of many parallel activities such as robots, conveyor belts, sensors, actuators and so on) and the computer’s display devices, the operator’s console, the database and the real-time clock. Fortunately, the speed of a modern computer is such that usually these actions may be carried out in sequence but give the illusion of being simultaneous. In some embedded systems, however, this may not be the case, for example

where the data is to be collected and processed at various geographically distributed sites, or where the response time of the individual components cannot be met by a single computer. In these cases, it is necessary to consider distributed and multiprocessor embedded systems.

A major problem associated with the production of software for systems which exhibit concurrency is how to express that concurrency in the structure of the program. One approach is to leave it all up to the programmer who must construct his/her system so that it involves the cyclic execution of a program sequence to handle the various concurrent tasks. There are several reasons, however, why this is inadvisable.

- It complicates the programmer's already difficult task and involves him or her in considerations of structures which are irrelevant to the control of the tasks in hand.
- The resulting program is more obscure and inelegant.
- It makes proving program correctness more difficult.
- It makes decomposition of the problem more complex.
- Parallel execution of the program on more than one processor is much more difficult to achieve.
- The placement of code to deal with faults is more problematic.

Older real-time programming languages, for example, RTL/2 and Coral 66, relied on operating system support for concurrency; and C is usually associated with Unix, Linux or POSIX. However, the more modern languages, such as Ada and Java, have direct support for general concurrent programming.

Although concurrency is a fundamental characteristic of real-time systems, different types of systems need different facilities. For reactive control systems with hard timing constraints, but relatively straightforward behaviour, it is sufficient to constrain each concurrent activity to be of the form:

```
input data
required computations with no external interactions
output data
```

For more complicated systems, the input and output activities cannot be separated from the computational part. Only during the computation activity itself will it be possible to determine the external data that is needed (for example from another activity, a shared database or from some interface into the environment).

In Chapters 4, 5 and 6, various models of concurrent programming are considered in detail. Attention is then focused, in the following two chapters, on achieving reliable communication and synchronization between concurrent processes in the presence of design errors.

1.3.3 Low-level programming

The nature of embedded systems requires the computer components to interact with the external world. They need to monitor sensors and control actuators for a wide variety of real-world devices. These devices interface to the computer via input and output registers, and their operational requirements are device- and computer-dependent. Devices may

also generate interrupts to signal to the processor that certain operations have been performed or that error conditions have arisen.

In the past, interfacing to devices has either been left under the control of the operating system, or has required the application programmer to resort to assembly language inserts to control and manipulate the registers and interrupts. Nowadays, because of the variety of devices and the time-critical nature of their associated interactions, their control must often be direct, and not through a layer of operating system functions. Furthermore, reliability requirements argue against the use of low-level programming techniques.

Since real-time systems are time-critical, efficiency of implementation will be more important than in other systems. It is interesting that one of the main benefits of using a high-level language is that it enables the programmer to abstract away from implementation details, and to concentrate on solving the problem at hand. Unfortunately, the embedded computer systems programmer cannot afford this luxury. He or she must be constantly concerned with the cost of using particular language features. For example, if a response to some input is required within a microsecond there is no point in using a language feature whose execution takes a millisecond!

It was noted earlier that a significant number of embedded systems are now mobile and rely on batteries for power. Efficient implementation can reduce the load on the processor, and memory, and lead to lowering of processor speed and the subsequent reduction in battery usage. Typically, halving of processor speed leads to a quadrupling of the life of the battery.

In Chapter 14, the facilities provided by real-time programming languages which enable the specification of device registers and interrupt control will be considered. The role of the execution environment in providing efficient and predictable implementations will also be examined.

1.3.4 Support for numerical computation

As was noted earlier, many real-time systems involve the control of some engineering activity. Figure 1.6 exemplifies a simple control system. The controlled entity, the plant, has a vector of output variables, y , that change over time, hence $y(t)$. These outputs are compared with the desired (or reference) signal $r(t)$ to produce an error signal, $e(t)$. The controller uses this error vector to change the input variables to the plant, $u(t)$. For a very simple system, the controller can be an analog device working on a continuous signal.

Figure 1.6 illustrates a feedback controller. This is the most common form, but feed-forward controllers are also used. In order to calculate what changes must be made

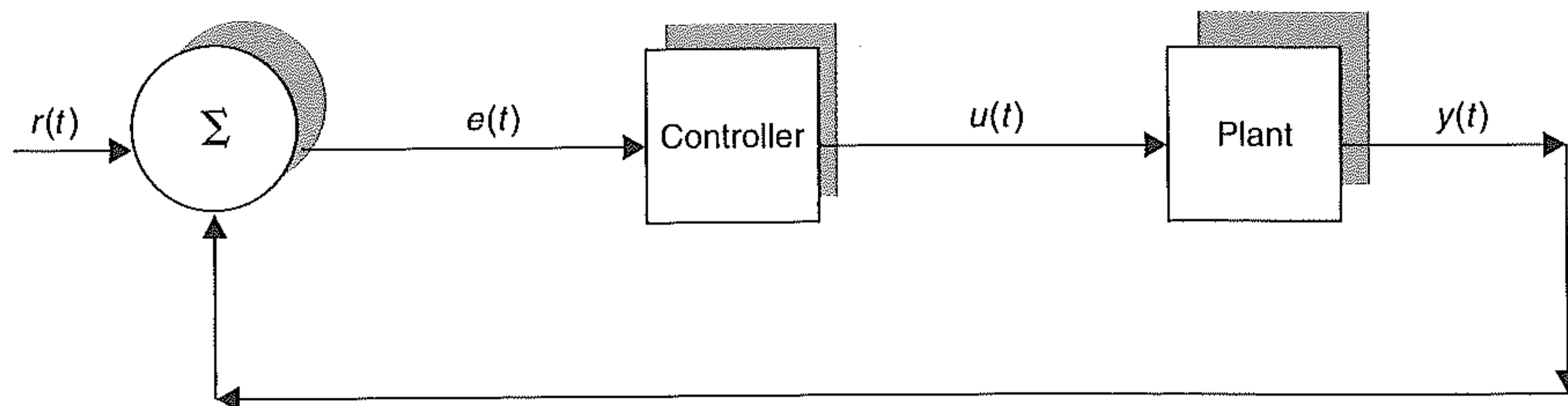


Figure 1.6 A simple controller.

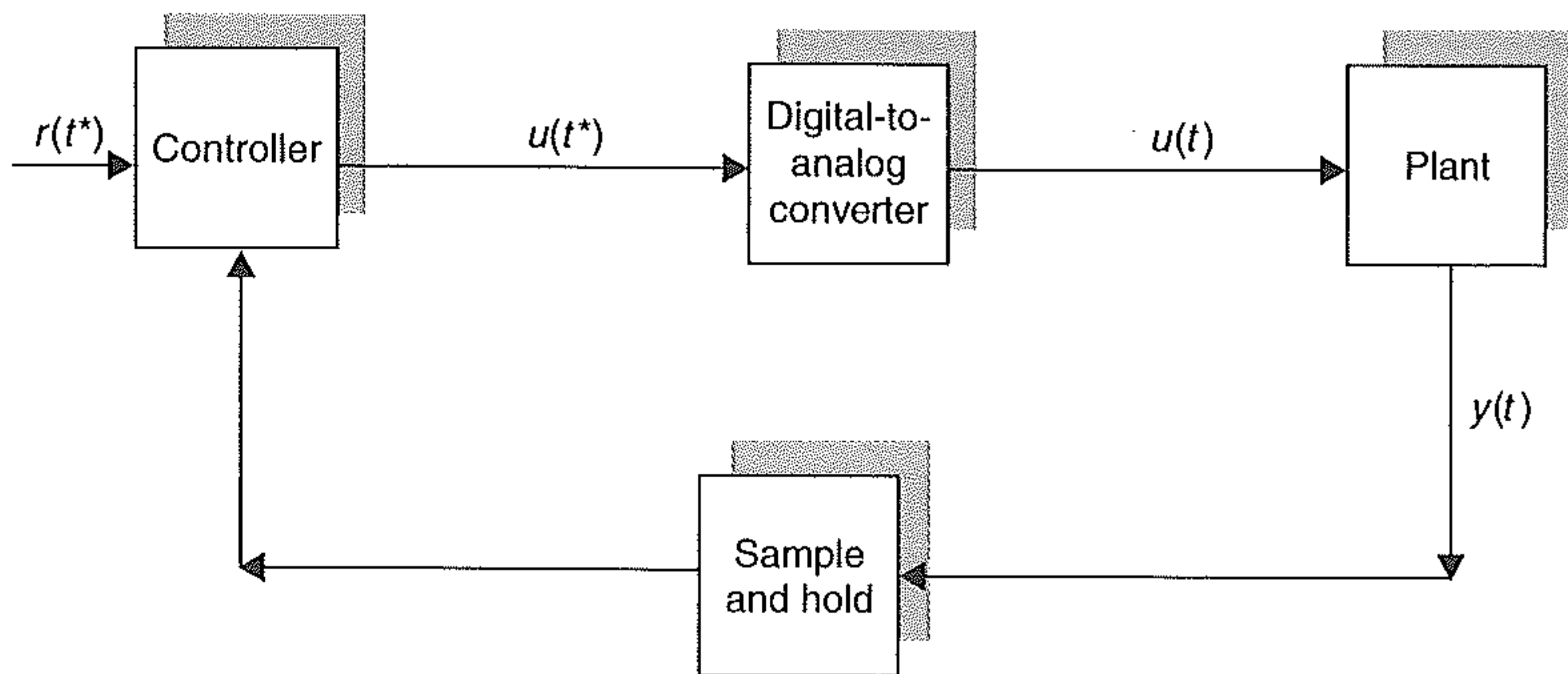


Figure 1.7 A simple computerized controller.

to the input variables, so that a desirable effect on the output vector takes place, it is necessary to have a mathematical model of the plant. The derivation of these models is the concern of the distinct discipline of control theory. Often a plant is modelled as a set of first-order differential equations. These link the output of the system with the internal state of the plant and its input variables. Changing the output of the plant involves solving these equations to give required input values. Most physical systems exhibit inertia, so that change is not instantaneous. A real-time requirement to move to a new set point within a fixed time period will add to the complexity of the manipulations needed, both to the mathematical model and to the physical system. The fact that, in reality, linear first-order equations are only an approximation to the actual characteristics of the system also presents complications.

Because of these difficulties, the complexity of the model, and the number of distinct (but not independent) inputs and outputs, most controllers are implemented as digital computers. The introduction of a digital component into the system changes the nature of the control cycle. Figure 1.7 is an adaptation of the earlier model. Items marked with a * are now discrete values; the sample and hold operation being carried out by an analog-to-digital converter, both converters being under the direct control of the computer.

Within the computer, the differential equations can be solved by numerical techniques, although the algorithms themselves need to be adapted to take into account the fact that plant outputs are now being sampled. The design of control algorithms is a topic outside the scope of this book; the implementation of these algorithms is, however, of direct concern. They can be mathematically complex and require a high degree of precision. A fundamental requirement of a real-time programming language, therefore, is the ability to manipulate real or floating-point numbers. Fortunately most engineering languages do provide the necessary abstractions in this area. There are standards for floating-point arithmetic that fully specify the required output of floating-point computations even when there are overflows, underflow, etc.

1.3.5 Large and complex

It is often said that most of the problems associated with developing software are those related to size and complexity. Writing small programs presents no significant problems

as they can be designed, coded, maintained and understood by a single person. If that person leaves the company or institution using the software, then someone else can learn the program in a relatively short period of time. Indeed, for these programs, there is an *art* or *craft* to their construction and *small is beautiful*.

Unfortunately, not all software exhibits this most desirable characteristic of smallness. Lehman and Belady (1985), in attempting to characterize large systems, reject the simple and perhaps intuitive notion that largeness is simply proportional to attributes such as: the number of instructions, lines of code or modules making up a program and to algorithmic complexity. Instead, they relate largeness to **variety**, and the degree of largeness to the amount of variety. Traditional indicators such as number of instructions and development effort are, therefore, just symptoms of variety.

The variety is that of needs and activities in the real world and their reflection in a program. But the real world is continuously changing. It is evolving. So too are, therefore, the needs and activities of society. Thus large programs, like all complex systems, must continuously evolve.

Embedded systems by their definition must respond to real-world events. The variety associated with these events must be catered for; the programs will, therefore, tend to exhibit the undesirable property of largeness. Inherent in the above definition of largeness is the notion of **continuous change**. The cost of redesigning or rewriting software to respond to the continuously changing requirements of the real world is prohibitive. Therefore real-time systems undergo constant maintenance and enhancements during their lifetimes. They must be extensible.

Although real-time software is often complex, features provided by real-time languages and environments enable these complex systems to be broken down into smaller components which can be managed effectively. The use of abstract data types, classes and objects, generic components, Application Program Interfaces (APIs) and interfaces, and separate compilation are all language features that engineering languages provide to manage this software complexity.

1.3.6 Extremely reliable and safe

The more society relinquishes control of its vital functions to computers, the more imperative it becomes that those computers do not fail. The failure of a system involved in automatic fund transfer between banks can lead to millions of dollars being lost irretrievably; a faulty component in electricity generation could result in the failure of a vital life-support system in an intensive care unit; and the premature shutdown of a chemical plant could cause expensive damage to equipment or environmental harm. These somewhat dramatic examples illustrate that computer hardware and software must be reliable and safe. Even in hostile environments, such as those found in military applications, it must be possible to design and implement systems that will fail only in a controlled way. Furthermore, where operator interaction is required, care must be taken in the design of the interface in order to minimize the possibility of human error.

The sheer size and complexity of real-time systems exacerbate the reliability problem; not only must expected difficulties inherent in the application be taken into account, but also those introduced by faulty software design.

In Chapters 2 and 3, the problems of producing reliable and safe software will be considered along with the facilities that languages have introduced to cope with both expected and unexpected error conditions. The issue is examined further in Chapters 7 and 13.

1.3.7 Structure of the book

The characteristics of real-time systems outlined in these short sections introduce the topics that are covered in this book. In Chapters 2 and 3, the problems of producing reliable and safe software will be considered along with the facilities that languages have introduced to cope with both expected and unexpected error conditions. The issue is examined further in Chapters 7 and 13.

In Chapters 4, 5 and 6, various models of concurrent programming are considered in detail. Attention is then focused, in the following two chapters, on achieving reliable communication and synchronization between concurrent processes in the presence of design errors. Support for real-time abstractions is discussed in Chapters 9, 10 and 11, together, in Chapter 12, with the language facilities that assist in the scheduling of time-critical operations. The techniques used to detect and recover from timing failures are considered in Chapter 13.

In Chapter 14, the facilities provided by real-time programming languages which enable the specification of device registers and interrupt control are considered, along with the role of the execution environment in providing efficient and predictable implementations.

Finally in Chapter 15 a case study, that brings together many of the key issues of the book, is developed.

The remainder of this introductory chapter outlines a number of other important issues whose detailed consideration is nevertheless beyond the scope of this book. In particular the development cycle (i.e. requirements specification, design, implementation and testing) is introduced and a number of general language topics are summarized. These discussions provide the backdrop to the detailed treatments in the following chapters and help deal with the size and complexity problems inherent in the development of most real-time systems.

1.4 Development cycle for real-time systems

Clearly, the most important stage in the development of any real-time system is the generation of a consistent design that satisfies an authoritative specification of requirements. In this, real-time systems are no different from other computer applications, although their overall scale often generates quite fundamental design problems. The discipline of software engineering is now widely accepted as the focus for the development of methods, tools and techniques aimed at ensuring that the software production process is manageable, and that reliable and correct programs are constructed. It is assumed here that readers are familiar with the basic tenets of software engineering and consideration is thus restricted to the particular problems and requirements furnished by real-time embedded systems. Even within this restriction, it is not possible to give a comprehensive account of the many design methodologies proposed. Issues of design *per se* are not the main focus of attention in this book. Rather, the investigation of language and operating

system primitives, which allow designs to be realized, is the central theme. Within this context, the languages Ada, (Real-Time) Java and C (with Real-Time POSIX) will be considered in detail. Readers should consult the further reading list at the end of this chapter for additional material on the design process.

Although almost all design approaches are top-down, they are built upon an understanding of what is feasible at lower levels. In essence, all design methods involve a series of transformations from the initial statement of requirements to the executing code. This section gives a brief overview of some of the typical stages that are passed through on this route, that is:

- **requirements specification** – during which an authoritative specification of the system's required functional and meta-functional behaviour is produced;
- **architectural design** – during which a top-level description of the proposed system is developed;
- **detailed design** – during which the complete system design is specified;
- **coding** – during which the system is implemented;
- **testing** – during which the efficacy of the system is tested.

As different activities are isolated, notations are required that enable each stage to be documented. Transformations from one stage to another are, therefore, nothing more than translations from one notation to another. For example, a compiler produces executable code from source code that is expressed in a programming language. Unfortunately, other translations (further up the design hierarchy) are less well defined; usually because the notations employed are too vague and imprecise, and cannot fully capture the semantics of the requirements or of the design.

Linked to notations are the models that can be expressed in these languages. The current emphasis on development approaches is focused on what is called **model driven architectures** (MDA). Here formal notations are used to develop models of the systems that can then be subject to verification. Techniques such as model-checking and mechanical proof are used to increase the designer's confidence that the systems once fully implemented will behave as expected in both the functional and temporal domains. Advocates of the MDA approach argue that it will be possible to automatically generate the executable code of the implementation from these higher-level models. This may well be the right approach for the future for relatively straightforward systems. However, the current state of these MDA approaches is not sufficiently advanced for general real-time systems to be developed by code-generation techniques. Hence in this book the focus is on the abstractions and facilities provided by real-time programming languages.

1.4.1 Requirement specification

Almost all computing projects start with an informal description of what is desired. This should then be followed by an extensive analysis of requirements. It is at this stage that the functionality of the system is defined. In terms of specific real-time factors, the temporal behaviour of the system should be made quite explicit, as should the reliability requirements and the desired behaviour of the software in the event of component failure. The requirements phase will also define which acceptance tests should apply to the software.

In addition to the system itself, it is necessary to build a model of the environment of the application. It is a characteristic of real-time systems that they have important interactions with their environment. Hence such issues as maximum rate of interrupts, maximum number of dynamic external objects (for example, aeroplanes in an air traffic control system) and failure modes are all important.

The analysis phase provides an authoritative specification of requirements. It is from this that the design will emerge. There is no more critical phase in the software life cycle, and yet natural language documents are still the normal notation for this specification. Nevertheless, formal methods can be applied to these specifications and are increasing being required in safety-critical applications.

1.4.2 Design activities

The design of a large embedded system cannot be undertaken in one exercise. It must be structured in some way. To manage the development of complex real-time systems, two complementary approaches are often used: decomposition and abstraction. Together, they form the basis of most software engineering methods. Decomposition, as its name suggests, involves the systematic breakdown of the complex system into smaller and smaller parts until components are isolated that can be understood and engineered by individuals or small groups. At each level of decomposition, there should be an appropriate level of description and a method of documenting (expressing) this description. Abstraction enables the consideration of detail, particularly that appertaining to implementation, to be postponed. This allows a simplified view of the system and of the objects contained within it to be taken, which nevertheless still contains the essential properties and features. The use of abstraction and decomposition pervades the entire engineering process and has influenced the design of real-time programming languages and associated software design methods.

If a formal notation is used for the requirement specification then top-level designs may use the same notation and can thus be proven to meet the specification. Many structured notations are, however, advocated either to fill out the top-level design or to replace the formal notation altogether. Indeed, a structured top-level design may, in effect, be the authoritative specification of requirements.

The hierarchical development of software leads to the specification and subsequent development of program subcomponents. The needs of abstraction dictate that these subcomponents should have well-defined roles, and clear and unambiguous interconnections and interfaces. If the specification of the entire software system can be verified just in terms of the specification of the immediate subcomponents then decomposition is said to be **compositional**. This is an important property when formally analysing programs.

Sequential programs are particularly amenable to compositional methods, and a number of techniques have been used to encapsulate and represent subcomponents. Simula introduced the significant **class** construct. More recently *object-oriented* languages, such as C++, Java and Eiffel, have emerged to build upon the class construct. Ada uses a combination of packages, type extensions and interfaces to support object-oriented programming.

Objects, while providing an abstract interface, require extra facilities if they are to be used in a concurrent environment. Typically, this involves the addition of some

form of task (process). The **task** abstraction is, therefore, the abstraction on which this book will focus. In Chapter 4, the notion of task is introduced, Chapter 5 then considers shared-variable based tasks interaction. A more controlled and abstract interface is, however, provided by message-based process communication. This is discussed in Chapter 6.

Both object and task abstractions are important in the design and implementation of reliable embedded systems. These forms of encapsulation lead to the use of modules with well-defined (and abstract) interfaces. From the definition of modules, more sizeable *components* can be defined that may even be re-usable in subsequent designs. But how should a large system be decomposed into components and modules? To a large extent, the answer to this question lies at the heart of all software design activities. Cohesion and coupling are two metrics that are often used to describe the relationships between entities within a design (in the following, the term ‘module’ is used for a distinct software entity).

Cohesion is concerned with how well a module holds together – its internal strength. Allworth and Zobel (1987) give six measures of cohesion that range from the very poor to the strong.

- **Coincidental** – elements of the module are not linked other than in a very superficial way; for example, written in the same month.
- **Logical** – elements of the module are related in terms of the overall system, but not in terms of the actual software; for example all output device drivers.
- **Temporal** – elements of the module are executed at similar times; for example, start-up routines.
- **Procedural** – elements of the module are used together in the same section of the program; for example, user interface components.
- **Communicational** (*sic*) – elements of the module work on the same data structure; for example, algorithms used to analyse an input signal.
- **Functional** – elements of the module work together to contribute to the performance of a single system function; for example, the provision of a distributed file system.

Coupling, by comparison, is a measure of the interdependence of program modules. If two modules pass control information between them, they are said to possess high (or tight) coupling. Alternatively, the coupling is loose if only data is communicated. Another way of looking at coupling is to consider how easy it would be to remove a module (from a completed system) and replace it with an alternative one.

Within all design methods, a good decomposition is one that has *strong* cohesion and *loose* coupling. This principle is equally true in sequential and concurrent programming domains.

It was noted earlier that most real-time practitioners advocate the use of object and task (process) abstractions. Formal techniques do exist that enable concurrent time-constrained systems to be specified and analysed. Nevertheless, these techniques are not yet sufficiently mature to constitute ‘tried and tested’ design methods. Rather, the real-time industry uses, at best, structured methods and software engineering approaches that are applicable to all information processing systems. They do not give specific

support to the real-time domain, and they lack the richness that is needed if the full power of implementation languages is to be exploited.

1.4.3 Testing and simulation

With the high reliability requirements that are the essence of most real-time systems, it is clear that testing must be extremely stringent. A comprehensive strategy for testing involves many techniques, most of which are applicable to all software products. It is, therefore, assumed that the reader is familiar with these techniques.

The difficulty with real-time concurrent programs is that the most intractable system errors are usually the result of subtle interactions between tasks. Often the errors are also time-dependent and will only manifest themselves in rare states. Murphy's Law dictates that these rare states are also crucially important and only occur when the controlled system is, in some sense, critical.

Testing is, of course, not restricted to the final assembled system. The decomposition incorporated in the design and manifest within program modules (including tasks) forms a natural architecture for component testing. Of particular importance (and difficulty) within real-time systems is that not only must correct behaviour in a correct environment be tested, but dependable behaviour in an arbitrarily incorrect environment must be catered for. All error recovery paths must be exercised and the effects of simultaneous errors investigated.

To assist in any complex testing activity, a realistic test bed presents many attractions. For software, such a test environment is called a simulator.

A simulator is a program which imitates the actions of the engineering system in which the real-time software is embedded. It simulates the generation of interrupts and performs other I/O actions in real-time. Using a simulator, abnormal as well as 'normal' system behaviour can be created. Even when the final system has been completed, certain error states may only be safely experimented with via a simulator. The meltdown of a nuclear reactor is an obvious example.

Simulators are able to reproduce accurately the sequence of events expected in the real system. In addition, they can repeat experiments in a way that is usually impossible in a live operation. However, to faithfully recreate simultaneous actions it may be necessary to have a very powerful computational platform. And even then with very complicated applications it may not be possible to build an appropriate emulation of the system.

Simulators are clearly non-trivial and expensive systems to develop. They may even require special hardware. In the NASA Space Shuttle project, the simulators cost more than the real-time software itself. This money turned out to be well spent, with many system errors being found during hours of simulator 'flight'.

1.4.4 Postscript

In many ways, this discussion on design issues has been a divergent one. It has introduced more problem areas and engineering issues than can possibly be tackled in just one book. This broad sweep across the 'design process' is aimed at setting the rest of the book in context. By now focusing on language issues and programming activities, the reader will be able to understand the 'end product' of design, and judge to what extent current methodologies, techniques and tools are appropriate.

1.5 Languages for programming real-time systems

An important plateau between the top-level requirements specification and the executing machine code is the programming language. The development of implementation languages for real-time systems is the central theme of this book. Language design is still a very active research area. Although systems design should lead naturally into implementation, the expressive power of most modern languages is not matched by current design methodologies. Only by understanding what is possible at the implementation stage can appropriate design approaches be developed.

It is possible to identify three classes of programming languages which are, or have been, used in the development of real-time systems. These are assembly languages, sequential systems implementation languages and high-level concurrent languages. These types of languages will shortly be reviewed, but first some general language design criteria will be introduced.

1.5.1 General language design criteria

Although a real-time language may be designed primarily to meet the requirements of embedded computer system programming, its use is rarely limited to that area. Most real-time languages are also used as general-purpose systems implementation languages for applications such as compilers and operating systems.

Young (1982) lists the following six (sometimes conflicting) criteria as the basis of a real-time language design: security, readability, flexibility, simplicity, portability and efficiency. A similar list also appears in the original requirements for Ada.

Security

The security of a language design is a measure of the extent to which programming errors can be detected automatically by the compiler or language run-time support system. There is obviously a limit to the type and number of errors that can be detected by a language system; for example, errors in the programmer's logic cannot be detected automatically. A secure language must, therefore, be well structured and readable so that such errors can easily be spotted.

The benefits of security include:

- the detection of errors much earlier in the development of a program – generating an overall reduction in cost;
- compile-time checks have no overheads at run-time – a program is executed much more often than it is compiled.

The disadvantage of security is that it may result in a more complicated language with an increase in compilation time and compiler complexity.

Readability

The readability of a language depends on a variety of factors including the appropriate choice of keywords, the ability to define types and the facilities for program

modularization. As Young points out:

the aim is to provide a language notation with sufficient clarity to enable the main concepts of a particular program's operation to be assimilated easily by reading the program's text only, without resort to subsidiary flowcharts and written descriptions. (Young, 1982)

The benefits of good readability include:

- reduced documentation costs;
- increased security;
- increased maintainability.

The main disadvantage is that it usually increases the length of any given program.

Flexibility

A language must be sufficiently flexible to allow the programmer to express all the required operations in a straightforward and coherent fashion. Otherwise, as with older sequential languages, the programmer will often have to resort to operating system commands or machine code inserts to achieve the desired result.

Simplicity

Simplicity is a worthwhile aim of any design, be it of the international space station or a simple calculator. In programming languages, simplicity has the advantages of:

- minimizing the effort required to produce compilers;
- reducing the cost associated with programmer training;
- diminishing the possibility of making programming errors as a result of misinterpretation of the language features.

Flexibility and simplicity can also be related to the **expressive power** (the ability to express the solutions to a wide range of problems) and **usability** (ease of use) of the language.

Portability

A program, to a certain extent, should be independent of the hardware on which it executes. One of the main claims of Java is that programs are compiled once and run anywhere. For a real-time system, this is difficult to achieve (even with the advent of portable binary codes, such as Java Byte Code and ANDF (Venners, 1999; X/Open Company Ltd, 1996)), as a substantial part of any program will normally involve manipulation of hardware resources. However, a language must be capable of isolating the *machine-dependent* part of a program from the *machine-independent* part.

Efficiency

In a real-time system, response times must be guaranteed; therefore the language must allow efficient and predictable programs to be produced. Mechanisms which lead to unpredictable run-time overheads should be avoided. Obviously, efficiency requirements must be balanced against security, flexibility and readability requirements.

1.5.2 Assembly languages

Initially, most real-time systems were programmed in the assembly language of the embedded computer. This was mainly because high-level programming languages were not well supported on most microcomputers and assembly language programming appeared to be the only way of achieving efficient implementations that could access hardware resources.

The main problem with the use of assembly languages is that they are machine-oriented rather than problem-oriented. The programmer can become encumbered with details which are unrelated to the algorithms being programmed, with the result that the algorithms themselves become obscure. This keeps development costs high and makes it very difficult to modify programs when errors are found or enhancements required.

Further difficulties arise because programs cannot be moved from one machine to another but must be rewritten. Also staff must be retrained if they are required to work with other machines.

1.5.3 Sequential systems implementation languages

As computers became more powerful, programming languages more mature, and compiler technology progressed, the advantages of writing real-time software in a high-level language outweighed the disadvantages. To cope with deficiencies in languages like FORTRAN, new languages were developed specifically for embedded programming. In the United States Air Force, for example, Jovial was in common use. In the UK, the Ministry of Defence standardized on Coral 66, and large industrial concerns like ICI standardized on RTL/2. Currently, the C and C++ programming languages are popular.

All these languages have one thing in common – they are sequential. They also tend to be weak in the facilities they provide for real-time control and reliability. As a result of these shortcomings, it is often necessary to rely on operating system support and assembly code inserts.

1.5.4 High-level concurrent programming languages

In spite of the increasing use of application-tailored languages, the production of computer software became progressively more difficult during the 1970s as computer-based systems became larger and more sophisticated. These problems grew to what became known as the *software crisis*. There are several symptoms of this crisis which have been recognized (Booch, 1986).

- **Responsiveness** – production systems which have been automated often do not meet users' needs.

- **Reliability** – software is unreliable and will often fail to perform to its specification.
- **Cost** – software costs are seldom predictable.
- **Modifiability** – software maintenance is complex, costly and error prone.
- **Timeliness** – software is often delivered late.
- **Transportability** – software in one system is seldom used in another.
- **Efficiency** – software development efforts do not make optimal use of the resources involved.

Perhaps one of the best illustrations of the impact of the software crisis can be found in the American Department of Defense's (DoD) search for a common high-order programming language for all its applications. As hardware prices began to fall during the 1970s, the DoD's attention was focused on the rising cost of its embedded software. It estimated that, in 1973, three thousand million dollars were spent on software alone. A survey of programming languages showed that at least 450 general-purpose programming languages and incompatible dialects were used in DoD embedded computer applications. An evaluation of existing languages occurred in 1976 against an emerging set of requirements. These evaluations resulted in four main conclusions (Whitaker, 1978):

- (1) No current language was suitable.
- (2) A single language was a desirable goal.
- (3) The state-of-the-art of language design could meet the requirements.
- (4) Development should start from a suitable language base; those recommended were Pascal, PL/I and Algol 68.

The result was the birth of a new language in 1983 called Ada. In 1995, the language was updated to reflect 10 years of use and modern advances in programming language design. The same occurred in 2005 when a number of key features were added to Ada.

Other older languages of note include PEARL, used extensively in Germany for process control applications, Mesa (Xerox Corporation, 1985), used by Xerox in their office automation equipment, and CHILL (CCITT, 1980) which was developed in response to CCITT requirements for programming telecommunication applications.

With the advent of the Internet, the Java programming language has become popular. Although initially not suitable for real-time programming, recently much effort has been dedicated to producing real-time versions of Java – this will be discussed at length later in the book.

Summary

In this chapter, a real-time system has been defined as:

any information processing activity or system which has to respond to externally generated input stimuli within a finite and specified delay.

Two main classes of such systems have been identified: hard real-time systems, where it is absolutely imperative that responses occur within the specified

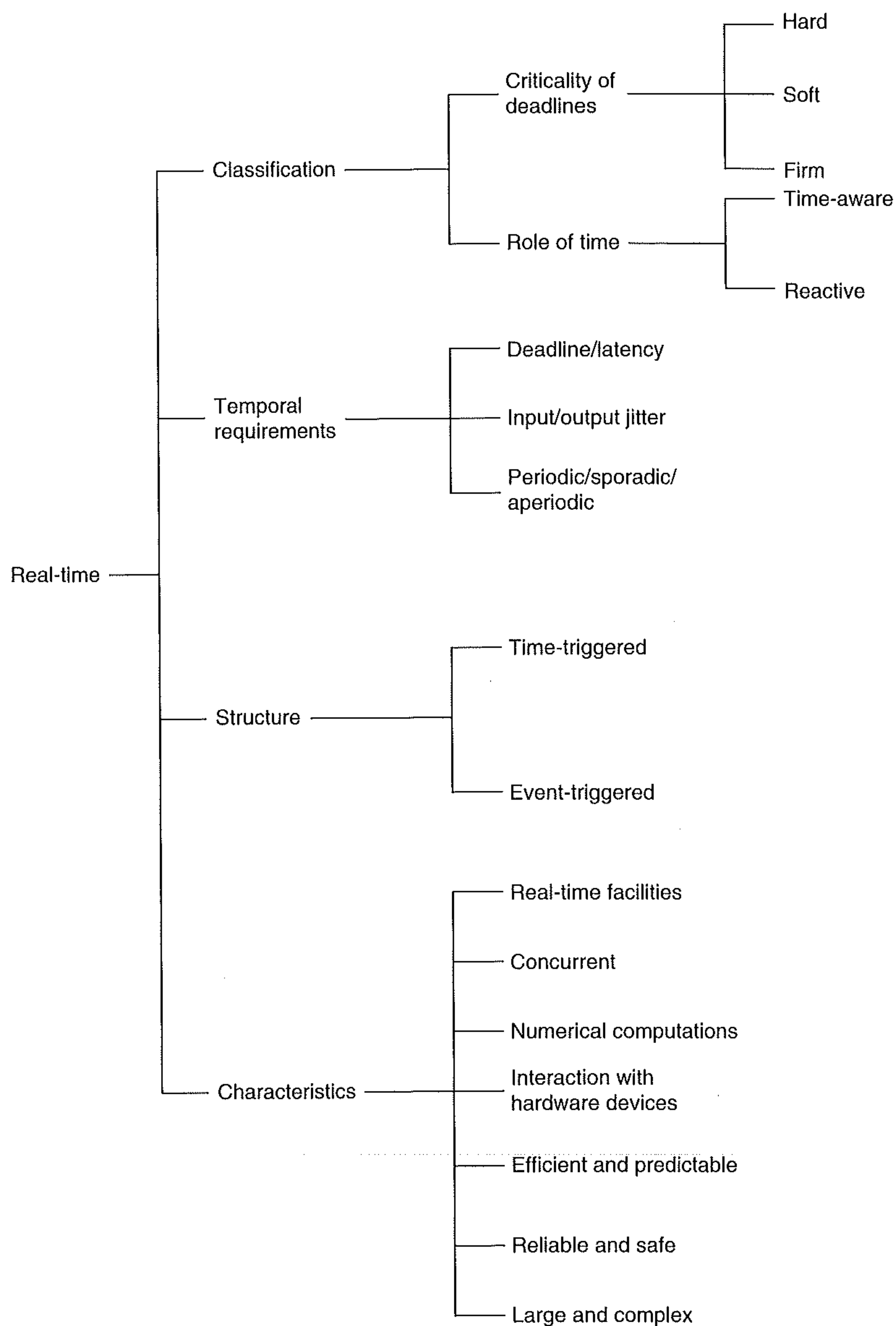


Figure 1.8 Aspects of real-time systems.

deadline; and soft real-time systems, where response times are important, but the system will still function correctly if deadlines are occasionally missed. Various types of real-time systems have been introduced including reactive systems and time-aware systems, also time-triggered and event-triggered systems.

The basic characteristics of a general real-time or embedded computer system have been considered. They are:

- real-time control;
- concurrent control of separate system components;
- low-level programming;
- support for numerical computation;
- largeness and complexity;
- extreme reliability and safety.

The main aspects associated with the term 'real-time' that have been introduced in this chapter are illustrated in Figure 1.8 opposite.

This chapter has also outlined the major stages involved in the design and implementation of real-time systems. These include requirements specification, systems design, detailed design, coding and testing. The high reliability requirements of real-time systems dictate that, wherever possible, rigorous methods should be employed.

Implementation, which is the primary focus of attention in this book, necessitates the use of a programming language. Early real-time languages lacked the expressive power to deal adequately with this application domain. More recent languages have attempted to incorporate concurrency and error-handling facilities. A discussion of these features is contained in subsequent chapters. The following general criteria were considered a useful basis for a real-time language design: security, readability, flexibility, simplicity, portability and efficiency.

Further reading

- Bailey, D. L. and Buhr, R. J. A. (1998) *Introduction to Real-Time Systems: From Design to Networking with C/C++*. Upper Saddle River, NJ: Prentice Hall.
- Booch, G., Rumbaugh, J. and Jacobson, I. (1999) *The Unified Modeling Language User Guide*. Harlow: Addison-Wesley.
- Burns, A. and Wellings, A. J. (1995) *Hard Real-Time HOOD: A Structured Design Method for Hard Real-Time Ada Systems*. New York: Elsevier.
- Cooling, J. E. (1995) *Software Design for Real-Time Systems*. London: International Thompson Computer Press.
- Douglass, B. P. (1999) *Doing Hard Time: Developing Real-Time Systems with UML, Objects, Frameworks and Patterns*. Harlow: Addison-Wesley.
- Gomaa, H. (2000) *Designing Concurrent, Distributed and Real-Time Applications in UML*. Reading, MA: Addison-Wesley.
- Jacobson, I., Booch, G. and Rumbaugh, J. (1999) *The Unified Software Development Process*. Harlow: Addison Wesley Longman.
- Koptez, H. (1997) *Real-time Systems*. New York: Kluwer Academic.

- Laplante, P. (1997) *Design and Application of Real-Time Systems*. New York: Institute of Electrical & Electronic Engineers.
- Liu, J. W. S. (2000) *Real-Time Systems*. New York: Prentice Hall.
- Schneider, S. (1998) *Concurrent and Real-time Systems*. New York: Wiley.
- Schneider, S. (1999) *Concurrent and Real-time Systems: The CSP Approach*. New York: Wiley.

Exercises

- 1.1 To what extent should the choice of a design method for real-time systems be influenced by:
- (a) likely implementation language
 - (b) support tools
 - (c) reliability requirements of the application
 - (d) training requirements of staff
 - (e) marketing considerations
 - (f) previous experiences
 - (g) cost?
- 1.2 In addition to the criteria given in this chapter, what other factors could be used in assessing programming languages?
- 1.3 At what stage in the design process should the views of the end-user be obtained?
- 1.4 Should software engineers be liable for the consequences of faulty real-time systems?
- 1.5 New medicines cannot be introduced until appropriate tests and trials have been carried out. Should real-time systems be subject to similar legislation? If a proposed application is too complicated to simulate, should it be constructed?
- 1.6 Should the Ada language be the only language used in the implementation of embedded real-time systems?
- 1.7 To what extent does UML allow hard real-time systems to be designed and analysed?